

Kernels and SMs

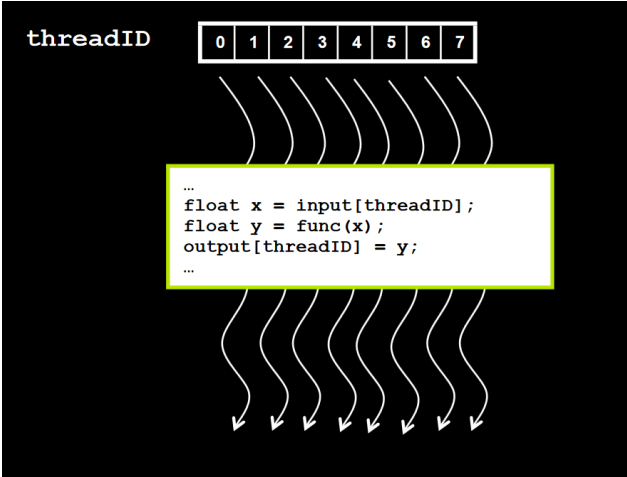
[Understanding GPU Architecture](#) > [GPU Characteristics](#) > [Kernels and SMs](#)

We continue our survey of GPU-related terminology by looking at the relationship between kernels, thread blocks, and streaming multiprocessors (SMs).

Kernels (in software)

A function that is meant to be executed in parallel on an attached GPU is called a *kernel*. In CUDA, a kernel is usually identified by the presence of the `__global__` specifier in front of an otherwise normal-looking C++ function declaration. The designation `__global__` means the kernel may be called from either the host or the device, but it will execute on the device.

Instead of being executed only once, a kernel is executed N times in parallel by N different threads on the GPU. Each thread is assigned a unique ID (in effect, an index) that it can use to compute memory addresses and make control decisions.



Showing how a CUDA kernel is executed by an array of threads.
Source: [NVIDIA CUDA Zone](#)

Accordingly, kernel calls must supply special arguments specifying how many threads to use on the GPU. They do this using CUDA's "execution configuration" syntax, which looks like this: `fun<<<1, N>>>(x, y, z)`. Note that the first entry in the configuration (1, in this case) gives the number of *blocks* of N threads that will be launched.

Streaming multiprocessors (in hardware)

On the GPU, a kernel call is executed by one or more streaming multiprocessors, or SMs. The SMs are the hardware homes of the CUDA cores that execute the threads. The CUDA cores in each SM are always arranged in sets of 32 so that the SM can use them to execute full *warps* of threads. The exact number of SMs available in a device depends on its NVIDIA processor family (Volta, Turing, etc.), as well as the specific model number of the processor. Thus, the Volta chip in the Tesla V100 has 80 SMs in total, while the more recent Turing chip in the Quadro RTX 5000 has just 48.

However, the number of SMs that the GPU will actually use to execute a kernel call is limited to the number of thread *blocks* specified in the call. Taking the call `fun<<<M, N>>>(x, y, z)` as an example, there are at most M blocks that can be assigned to different SMs. A thread block may not be split between different SMs. (If there are more blocks than available SMs, then more than one block may be assigned to the same SM.) By distributing blocks in this manner, the GPU can run independent blocks of threads in parallel on different SMs.

Each SM then divides the N threads in its current block into warps of 32 threads for parallel execution internally. On every *cycle*, each SM's *schedulers* are responsible for assigning full warps of threads to run on available sets of 32 CUDA cores. (The Volta architecture has 4 such schedulers per SM.) Any leftover, partial warps in a thread block will still be assigned to run on a set of 32 CUDA cores.

The SM includes several levels of memory that can be accessed only by the CUDA cores of that SM: *registers*, *L1 cache*, constant caches, and shared memory. The exact properties of the per-SM and global memory available in Volta GPUs will be outlined shortly.